

Chat con Pedro Morales Pino

Documentación de desarrollo — Single Page Application con Google Gemini AI

Proyecto Integrador — Módulo 3

Full stack developer — Henry

Autor: Sebastián Pérez Rodríguez

Repositorio: github.com/sebastianperodriguez/Chat-Pedro-Morales-Pino-PI-M3

Aplicación desplegada: <https://chat-pedro-morales-pino-pi-m3.vercel.app/>

Fecha: 02/07/2026

Contenido

- 1. Introducción y objetivo del proyecto
- 2. El personaje: Pedro Morales Pino
- 3. Arquitectura y stack tecnológico
- 4. Proceso de desarrollo (flujo de trabajo)
- 5. Diseño responsive — evidencia en 3 tamaños
- 6. Pruebas unitarias con Vitest
- 7. Despliegue en Vercel
- 8. Registro y evidencia del uso de Inteligencia Artificial
- 9. Conclusiones

1. Introducción y objetivo del proyecto

Este documento describe el proceso de desarrollo de una Single Page Application (SPA) que permite chatear con un personaje histórico —Pedro Morales Pino— utilizando Google Gemini AI como motor conversacional. El proyecto corresponde al entregable integrador del Módulo 3 de la carrera de programación en Henry, y su objetivo fue implementar routing básico entre tres vistas, un diseño responsive mobile-first, una integración segura con una IA generativa a través de funciones serverless de Vercel, y pruebas unitarias con Vitest.

El desarrollo se realizó de forma guiada: cada archivo del proyecto fue explicado paso a paso antes de ser escrito por el autor, con revisión y ejecución local en cada etapa (`npm run dev`, `vercel dev`, `npm test`) antes de avanzar a la siguiente. La sección 8 de este documento detalla ese proceso y aporta evidencia del uso de IA como herramienta de apoyo.

2. El personaje: Pedro Morales Pino

Pedro Morales Pino (Cartago, Valle del Cauca, 22 de febrero de 1863 – Bogotá, 4 de marzo de 1926) fue un compositor, bandolista y director musical colombiano, considerado el padre de la música colombiana. Nació en una familia humilde: de niño vendía dulces en la calle, lo que lo acercó a los músicos populares, y su madre le regaló su primer tiple. Más adelante estudió en la Academia Nacional de Música de Bogotá con el maestro Julio Quevedo.

Su aporte más importante fue llevar el bambuco, el pasillo y la danza —géneros de tradición oral— al pentagrama, con técnica académica, además de modernizar la bandola andina. En 1897 fundó la Lira Colombiana, una estudiantina con la que giró por Centroamérica y Estados Unidos, siendo uno de los primeros músicos colombianos en triunfar en el extranjero. Compuso más de 100 obras, entre ellas los pasillos "Joyeles", "Reflejos" y "Pierrot", y el bambuco "Cuatro preguntas".

Por qué se eligió este personaje

- Personalidad e historia real, con abundante documentación biográfica disponible, además del interés personal por difundir esta música e instrumentos musicales.
- Al ser una figura histórica de dominio público, no genera conflictos de derechos de autor sobre personajes de ficción.
- Su identidad cultural colombiana permitió construir una experiencia visual e idiomática coherente (paleta de "partitura antigua", tono de voz latinoamericano).

Diseño del system prompt

El system prompt (implementado en `api/chat.js`) fue construido a partir de datos biográficos verificados en fuentes como la Biblioteca Virtual del Banco de la República, Wikipedia, EcuRed y Radio Nacional de Colombia. Define el tono de voz del personaje (español latinoamericano cálido y

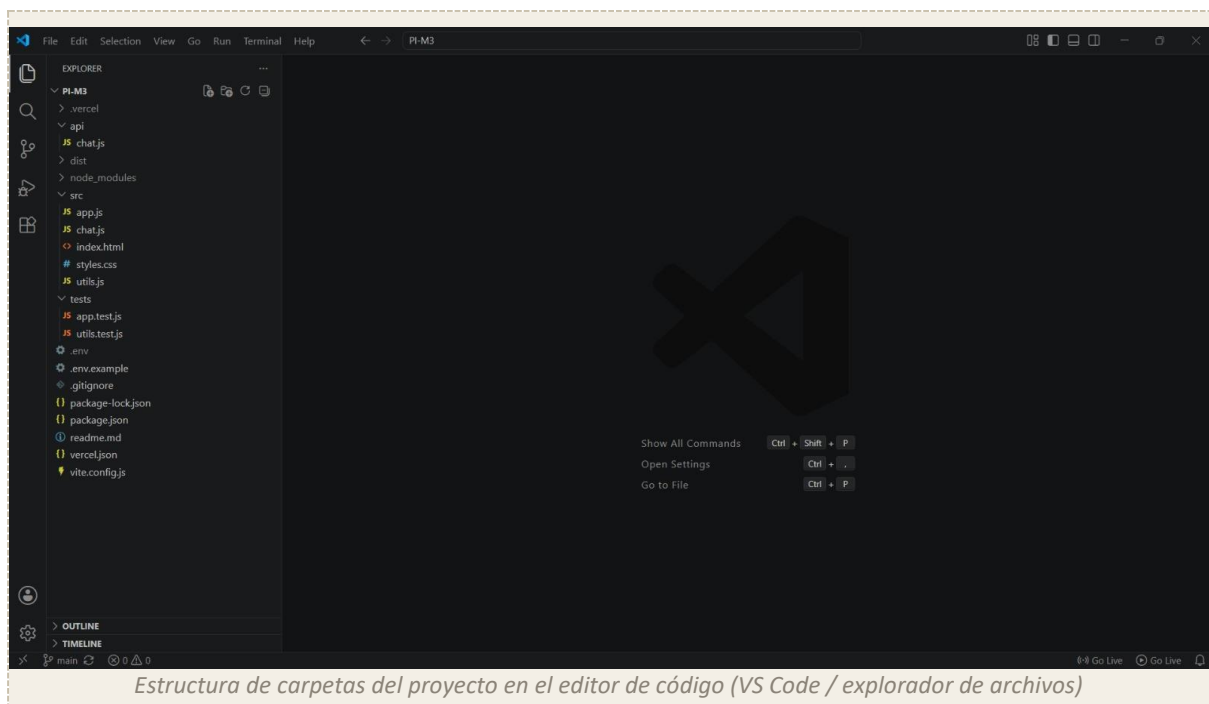
cercano), los temas que puede abordar (su vida, sus giras, sus composiciones), y restricciones de formato para que las respuestas sean breves y apropiadas para un chat.

3. Arquitectura y stack tecnológico

Stack

- Vite + JavaScript vanilla (sin frameworks) para el frontend.
- Routing propio implementado con la History API (pushState / popstate), sin recargar la página.
- Vercel Serverless Functions como puente seguro hacia la API de Google Gemini.
- Vitest + jsdom para las pruebas unitarias.

Estructura de carpetas



Flujo de datos del chat

El cliente nunca se comunica directamente con la API de Gemini. En su lugar, envía el mensaje del usuario y el historial de la conversación a la función serverless /api/chat, que agrega el system prompt del personaje, realiza la petición a Gemini usando la variable de entorno GEMINI_API_KEY (disponible solo en el servidor) y devuelve al cliente únicamente el texto de la respuesta. De esta forma la clave de la API jamás se expone en el navegador.

4. Proceso de desarrollo (flujo de trabajo)

El proyecto se construyó de forma incremental, archivo por archivo, en el siguiente orden:

- Configuración base: package.json, .gitignore, .env.example, vite.config.js.
- src/utils.js — funciones puras de routing, validación, formateo y parseo de datos.
- src/index.html y src/styles.css — estructura HTML y diseño visual mobile-first.
- src/app.js — router SPA con History API y las tres vistas (Home, Chat, About).
- src/chat.js — lógica de estado del chat, renderizado de mensajes y manejo de errores.
- api/chat.js — función serverless con el system prompt del personaje.
- tests/utils.test.js y tests/app.test.js — pruebas unitarias con Vitest.
- vercel.json — configuración de rewrites para que el routing de la SPA funcione en producción.
- Despliegue en Vercel con variables de entorno configuradas.

Vista Home

Página de bienvenida con la biografía resumida del personaje y botón de acceso al chat.



Vista Chat

Interfaz de conversación con diferenciación visual entre mensajes del usuario y del personaje, indicador de "escribiendo..." mientras se genera la respuesta, y scroll automático al último mensaje.

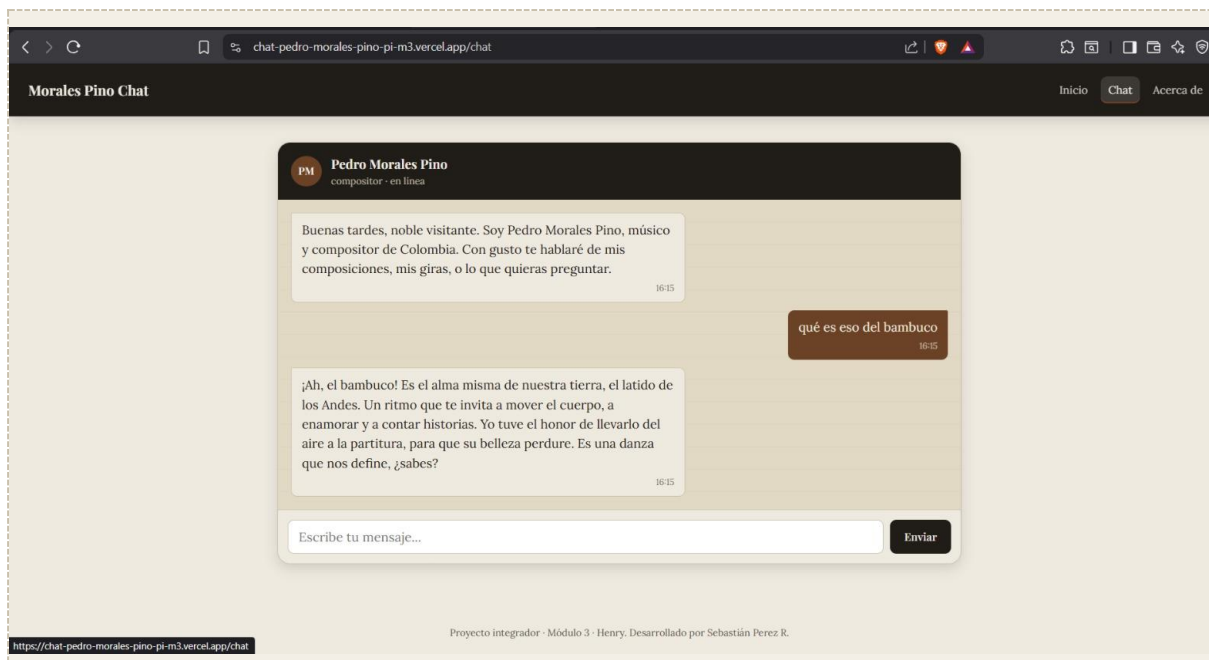
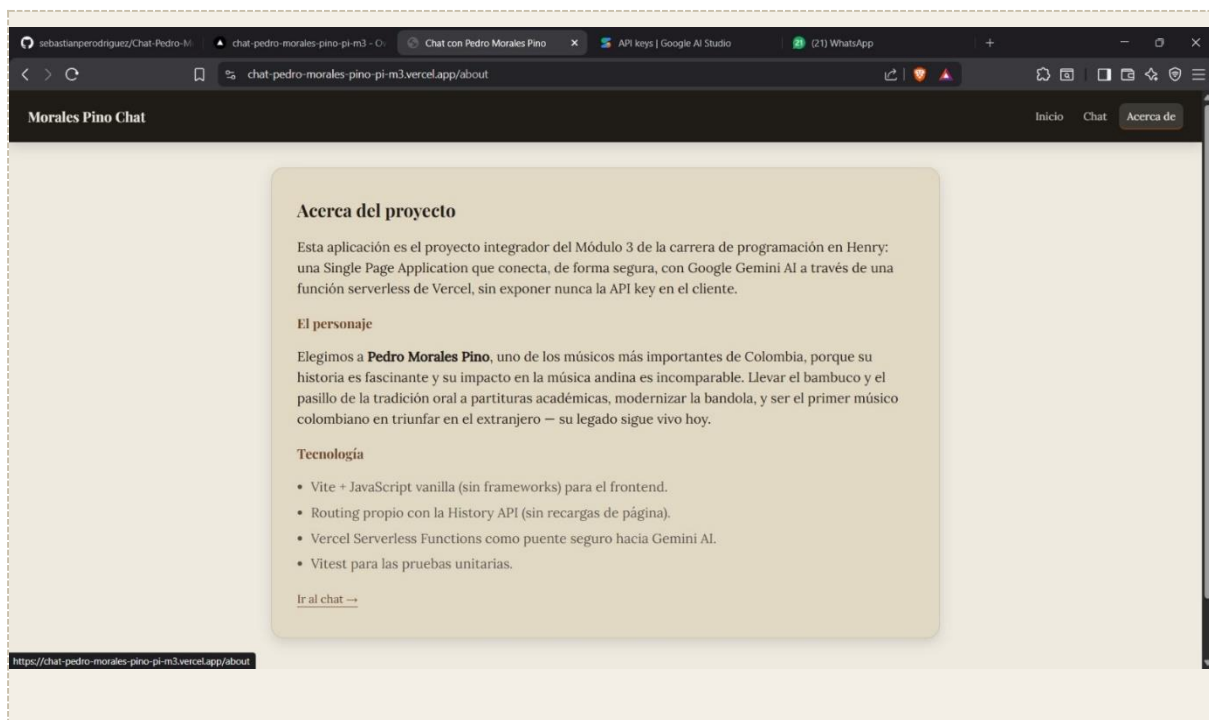


Figura 3. Vista Chat con una conversación de varios mensajes.

Vista About

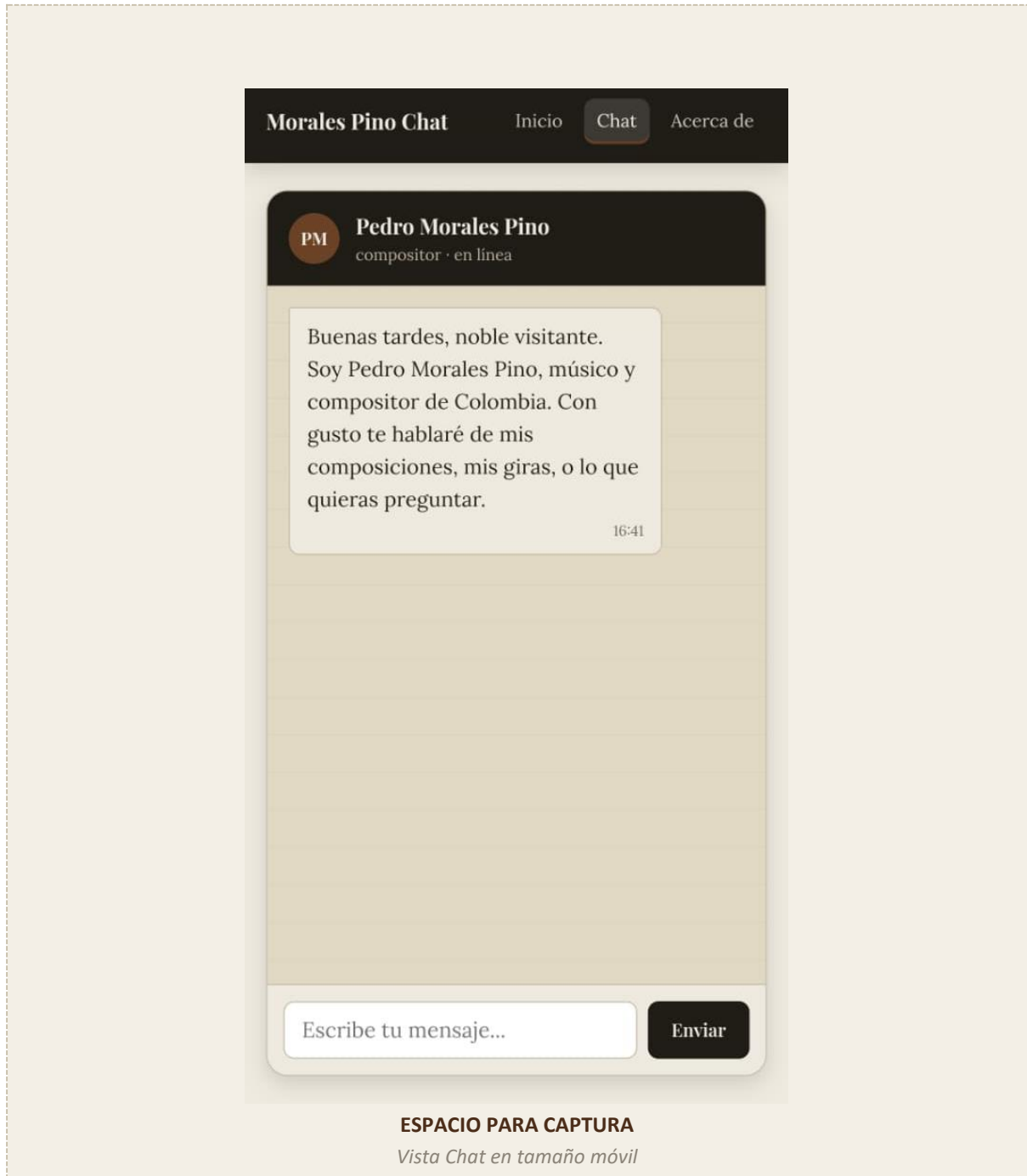
Información sobre el proyecto, el personaje y las tecnologías utilizadas.



Vista About

5. Diseño responsive — evidencia en 3 tamaños

El diseño se desarrolló mobile-first, con breakpoints en 640px (tablet) y 1024px (desktop). A continuación se muestra evidencia de la interfaz probada en DevTools del navegador en los tres tamaños definidos.

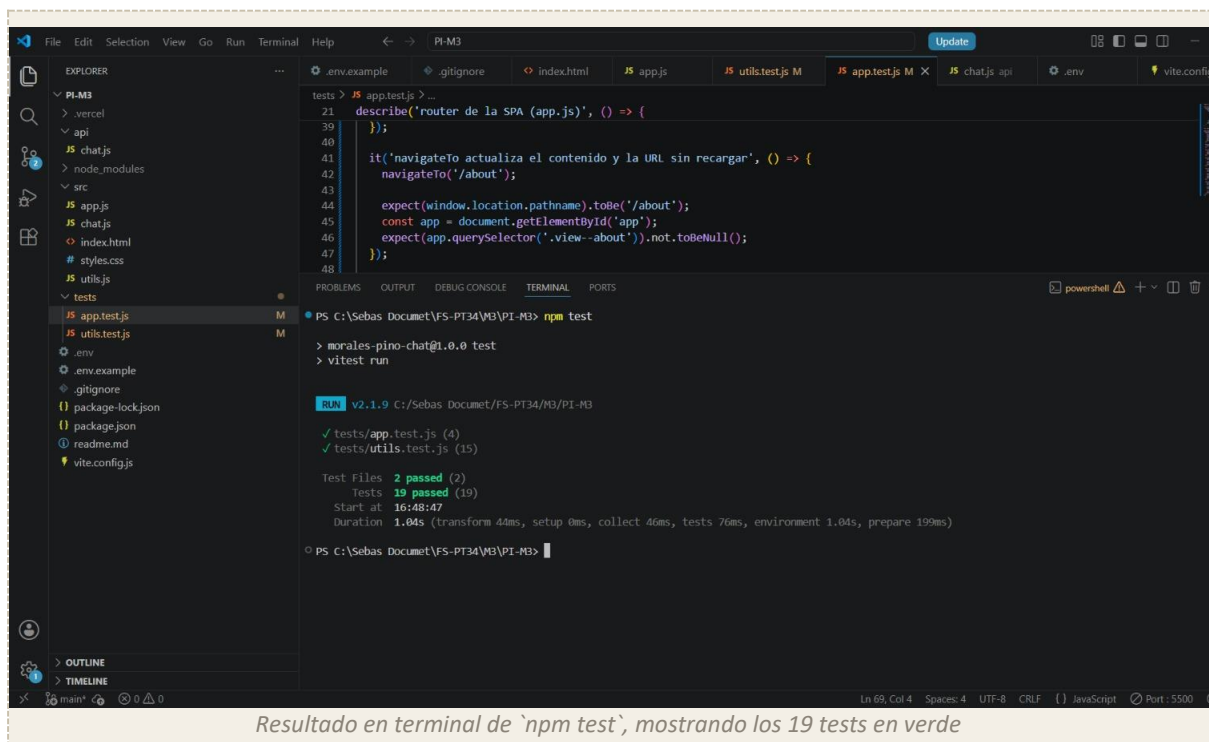


6. Pruebas unitarias con Vitest

Se implementaron 19 pruebas unitarias distribuidas en dos archivos:

- tests/utlis.test.js: valida las funciones puras — resolución de rutas, escapado de HTML, formato de hora, validación de mensajes, construcción del payload para Gemini y parseo de su respuesta.
- tests/app.test.js: valida el comportamiento del router SPA — vista por defecto, navegación programática, clics en enlaces con interceptación de History API, y soporte del botón atrás/adelante del navegador (popstate).

Comando de ejecución: npm test



The screenshot shows a VS Code editor with a file explorer on the left and a terminal at the bottom. The file explorer shows a project structure with files like .vercel, api, chat.js, node_modules, src, app.js, chat.js, index.html, styles.css, utlis.js, and tests. The tests directory contains app.test.js and utlis.test.js. The terminal shows the command `npm test` being executed, which runs vitest. The output shows that 2 test files passed (2 tests), with 19 tests passing in total. The duration of the tests is 1.04s.

```
tests > JS app.test.js > ...
21 describe('router de la SPA (app.js)', () => {
39   });
40
41   it('navigateTo actualiza el contenido y la URL sin recargar', () => {
42     navigateTo('/about');
43
44     expect(window.location.pathname).toBe('/about');
45     const app = document.getElementById('app');
46     expect(app.querySelector('.view--about')).not.toBeNull();
47   });
48 }
```

```
PS C:\Sebas Document\FS-PT34\M3\PI-M3> npm test
> morales-pino-chat@1.0.0 test
> vitest run

RUN v2.1.9 C:\Sebas Document\FS-PT34\M3\PI-M3
✓ tests/app.test.js (4)
✓ tests/utlis.test.js (15)

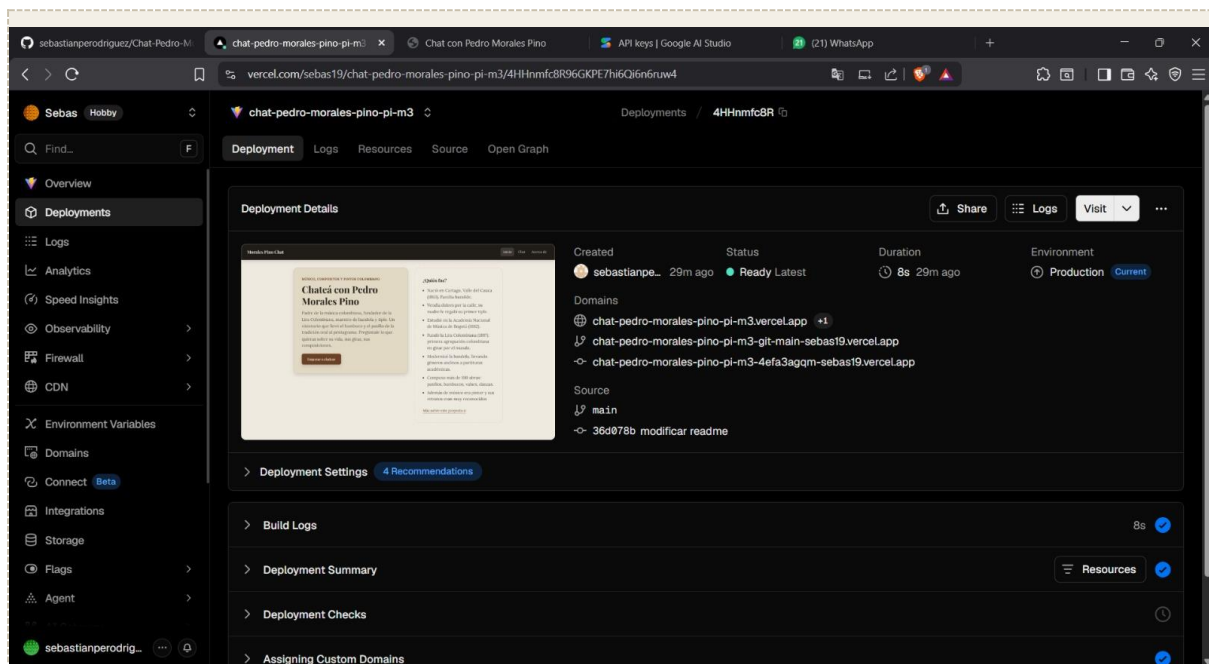
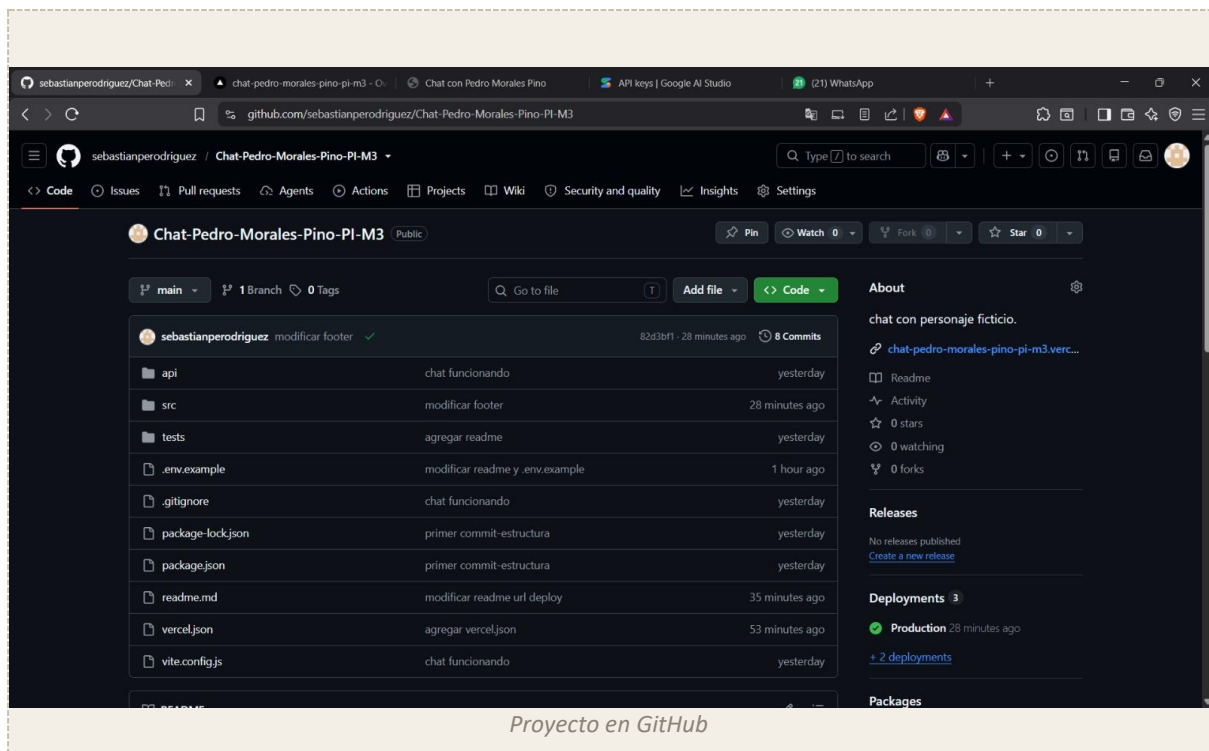
Test Files 2 passed (2)
Tests 19 passed (19)
Start at 16:48:47
Duration 1.04s (transform 44ms, setup 0ms, collect 46ms, tests 76ms, environment 1.04s, prepare 199ms)

PS C:\Sebas Document\FS-PT34\M3\PI-M3>
```

Resultado en terminal de `npm test`, mostrando los 19 tests en verde

7. Despliegue en Vercel

El proyecto se subió a un repositorio de GitHub y se conectó a Vercel para el despliegue continuo. Se configuraron las variables de entorno GEMINI_API_KEY y GEMINI_MODEL directamente en el dashboard de Vercel (nunca se suben al repositorio), y se ajustó el archivo vercel.json para que las reglas de rewrite permitieran el routing de la SPA sin romper los archivos estáticos (CSS, JS) ni las llamadas a /api.



8. Registro y evidencia del uso de Inteligencia Artificial

Se utilizó Claude (Anthropic) como tutor de código durante todo el desarrollo del proyecto, bajo una modalidad guiada: la IA explicaba la lógica y el propósito de cada función antes de que el autor la escribiera, en lugar de generar el proyecto completo de forma automática. A continuación se detalla en qué tareas específicas se utilizó:

- Definición de la arquitectura del proyecto (Vite + Vercel Functions + Vitest) y del orden de desarrollo archivo por archivo.
- Explicación línea por línea de las funciones de `src/utls.js` (routing, validación, formateo, construcción y parseo de payloads de la API de Gemini).
- Investigación de la biografía real de Pedro Morales Pino en fuentes verificables (Banrepcultural, Wikipedia, EcuRed, Radio Nacional de Colombia) para construir un system prompt fiel a los hechos históricos.
- Redacción y ajuste de tono del system prompt del personaje (de un registro inicial más castizo a un español latinoamericano natural, a pedido del autor).
- Redacción de los 19 tests unitarios de Vitest, con explicación de por qué la carga del módulo del router debía hacerse una sola vez (`beforeAll`) para evitar listeners duplicados en las pruebas.
- Depuración de errores durante el desarrollo: configuración de `appType: 'spa'` en Vite, corrección de un typo en `thinkingConfig` de la API de Gemini que causaba respuestas cortadas, diagnóstico de un error de API key inválida, y ajuste iterativo de las reglas de rewrites en `vercel.json` hasta lograr que no rompieran los estilos ni los archivos estáticos en local (`vercel dev`) ni en producción.
- Guía paso a paso para la conexión del repositorio de GitHub con Vercel y la configuración de variables de entorno en el dashboard.

Todo el código fue escrito, revisado y ejecutado por el autor del proyecto en cada etapa (`npm run dev`, `vercel dev`, `npm test`, `npm run build`) antes de continuar con el siguiente archivo, verificando su funcionamiento antes de hacer commit.

9. Conclusiones

El desarrollo de este proyecto permitió aplicar de forma práctica los conceptos de enrutamiento en aplicaciones de una sola página, diseño responsive mobile-first, integración segura con APIs de terceros mediante funciones serverless, y pruebas unitarias. El uso guiado de inteligencia artificial como herramienta de aprendizaje permitió comprender el porqué de cada decisión técnica —no solo copiar código— y resolver de forma metódica los errores encontrados durante el desarrollo local y el despliegue.

Checklist de requisitos cumplidos

- ☒ Rutas /home, /chat, /about con routing propio vía History API
- ☒ Diseño mobile-first con breakpoints probados en 3 tamaños
- ☒ Diferenciación visual entre mensajes de usuario y del personaje
- ☒ Estado de "escribiendo..." mientras la IA genera la respuesta
- ☒ Manejo de errores ante fallos de la API
- ☒ Historial de conversación mantenido en memoria durante la sesión
- ☒ Scroll automático al último mensaje
- ☒ API key de Gemini utilizada únicamente en el servidor
- ☒ 19 tests unitarios con Vitest
- ☒ Aplicación desplegada en Vercel con serverless functions funcionando